CO1 Scenario Based Questions : -

A. Computational Thinking – Decomposition, Abstraction & Pattern Recognition:

1. **Online Food Delivery System**
   You are asked to design an algorithm for an online food-delivery application. The application must handle user login, restaurant selection, food ordering, payment, and delivery tracking.
   Question: How would you decompose this large problem into smaller sub-problems?

2. **Hospital Management System**
   A hospital wants software to manage patients, doctors, appointments, billing, and medical reports.
   Question: What information would you keep as important abstractions, and what unnecessary details could be ignored while designing the algorithm?

3. **ATM Machine**
   An ATM allows users to withdraw money, deposit money, check balance, and change PIN.
   Question: How would you apply decomposition and abstraction to design the ATM system?

4. **Student Result Processing**
   A college has thousands of student marks and wants to calculate grades, class averages, toppers, and pass percentages.
   Question: Identify the smaller computational tasks involved and explain how you would decompose the problem.

5. **E-Commerce Search**
   An online shopping website allows users to search products by name, price, category, and rating.
   Question: What patterns might you identify from different search operations that could help you design an efficient algorithm?

B. Algorithm Design & Complexity:

1. **Finding a Student Record**
   A college stores student roll numbers in a sorted list containing 1 million records. You need to find one student's record quickly.
   Question: Would you use Linear Search or Binary Search? Why? What is the time complexity?
2. **Social Media Followers**
   You need to check whether a particular username exists among n usernames.
   Question: Compare Linear Search and Hash-based searching for this problem in terms of expected complexity.
3. **Library Book Arrangement**
   A library receives thousands of books with different accession numbers

and needs them arranged in ascending order.
Question: Which sorting algorithm would you choose if efficiency is important? Explain its complexity.

4. **Traffic Monitoring System**
   A traffic system receives vehicle IDs continuously and needs to determine whether a particular vehicle has already been recorded.
   Question: Which data-searching approach would you choose and why?

5. **Password Checking System**
   A system compares a user's entered password against stored information.
   Question: If the system performs n comparisons in the worst case, what is the time complexity? How could you reduce the number of operations?

C. Divide and Conquer:

1. **Large Student Dataset**
   You have 1 crore student records and need to sort them efficiently.
   Question: How can the Divide-and-Conquer strategy be applied? Identify the divide, conquer, and combine steps.

2. **Image Processing**
   An image contains millions of pixels. You want to perform an operation independently on different regions of the image.
   Question: How can divide-and-conquer be used to solve this problem?

3. **Finding Maximum Temperature**
   A weather station has temperature readings for a very large number of days.
   Question: Design a divide-and-conquer approach to find the maximum temperature.

4. **Searching a Sorted Database**
   A company has a sorted database containing millions of employee IDs.
   Question: Explain how Binary Search applies the divide-and-conquer concept. What is its time complexity?

5. **Merge Two Sorted Data Sources**
   Two departments have separately sorted employee lists. You need one completely sorted list.
   Question: How can the merge step of Merge Sort be useful in this scenario?

D. Dynamic Programming:

1. **Mobile Data Plans**
   A customer has several mobile plans with different costs and benefits and wants to select plans to maximize benefits within a fixed budget.
   Question: Would Dynamic Programming be suitable? Explain why.

2. **Delivery Route Optimization**
   A delivery company wants to find the cheapest route between multiple locations, where some smaller route calculations are repeatedly needed.
   Question: How can Dynamic Programming help avoid repeated computations?

3. **Staircase Problem**
   A person can climb either 1 or 2 stairs at a time. For n stairs, you need to calculate the number of possible ways to reach the top.
   Question: How would you solve this using Dynamic Programming? What repeated subproblem occurs?

4. **Employee Training Selection**
   A company has a limited training budget and several training programs with different costs and expected benefits.
   Question: How can the 0/1 Knapsack idea be applied to select the best combination?

5. **Text Comparison**
   A plagiarism detection system compares two documents and needs to identify the longest common sequence of characters/words.
   Question: Which Dynamic Programming problem is closely related to this scenario? Explain the idea.

E. Greedy Algorithms:

1. **ATM Currency Distribution**
   An ATM needs to provide ₹8,700 using the minimum possible number of notes.
   Question: How would a greedy strategy approach this problem? At every step, what choice would it make?

2. **Meeting Room Scheduling**
   A company has many meetings with different start and finish times but only one conference room.
   Question: How would you select the maximum number of non-overlapping meetings using a greedy strategy?

3. **Network Cable Installation**
   A company wants to connect several offices with minimum total cable cost.
   Question: Which greedy algorithm could be useful for this problem? Explain the basic idea.

4. **Emergency Delivery**
   A delivery vehicle has limited capacity and must prioritize deliveries based on a certain benefit-to-cost ratio.
   Question: How could a greedy strategy be used? What assumption must be true for the greedy choice to be appropriate?

5. **Internet Data Compression**
   A system needs to compress data by assigning shorter codes to frequently occurring characters.
   Question: Which greedy algorithm can be used? Explain why selecting the least-frequency elements repeatedly is useful.

F. Backtracking:

1. **N-Queens Problem**
   You need to place N queens on an N × N chessboard so that no two queens attack each other.
   Question: Explain how backtracking can solve this problem. What happens when a queen placement leads to an invalid configuration?

2. **Sudoku Solver**
   A Sudoku application needs to automatically fill an incomplete Sudoku puzzle.
   Question: How can backtracking be used? When should the algorithm backtrack?

3. **University Timetable**
   A university needs to assign subjects to classrooms and time slots while avoiding conflicts between subjects and teachers.
   Question: How can backtracking be applied to find a valid timetable?

G. Branch & Bound + Complexity:

1. **Travelling Salesperson Problem**
   A salesperson must visit 10 cities exactly once and return to the starting city while minimizing the total distance.
   Question: How can Branch and Bound reduce the number of possible routes that need to be explored? What is the purpose of a bound?

2. **Choosing an Algorithm for a Large Application**
   An e-commerce application needs to process millions of records. You have three algorithms with complexities:
3. **Algorithm A → O(n)**
4. **Algorithm B → O(n log n)**
5. **Algorithm C → O(n^2)**
6. **Question:**
   a) Which algorithm would generally be preferred for very large n?

b) Why is $O(n^2)$ potentially problematic?
c) Give a real-world situation where each complexity might occur.